LLM setup and troubleshooting

The provider integration is server-side in `hw_ai_support/llm.py`. Never place a provider key in the Vue code, browser storage, a screenshot, a command pasted into chat, or source control. If a key has been exposed, revoke it and configure a replacement.

Gemini hosted provider

Create a Gemini API key from Google AI Studio. Use a model name that is currently available to that key; availability can vary by account. Keep the key hidden while entering it in the Bench terminal:

```
cd ~/frappe-bench
read -rsp "Gemini API key: " GEMINI_KEY; echo
bench --site development.localhost set-config gemini_api_key "$GEMINI_KEY"
bench --site development.localhost set-config gemini_model "MODEL_AVAILABLE_TO_YOUR_KEY"
unset GEMINI_KEY
bench --site development.localhost clear-cache
```

Replace `MODEL_AVAILABLE_TO_YOUR_KEY` with the model ID listed for the key. In Desk, open HW AI Support Settings and set LLM Provider to `gemini`.

To list models using the key without putting it in the URL, enter it at the hidden prompt and pipe the response through Python:

```
read -rsp "Gemini API key: " GEMINI_KEY; echo
curl -sS --max-time 30 \
  -H "x-goog-api-key: $GEMINI_KEY" \
  "https://generativelanguage.googleapis.com/v1beta/models" \
| python3 -c 'import json,sys; d=json.load(sys.stdin); print("\n".join(m["name"] for m in d.get(
"models",[]) if "generateContent" in m.get("supportedGenerationMethods",[])); print(d.get("error",
""))'
unset GEMINI_KEY
```

Then verify the provider from the Bench:

```
bench --site development.localhost execute hw_ai_support.llm.ping
```

If the ping reports HTTP 400 and you need the exact provider message, compare it with a minimal direct request. Set `GEMINI_MODEL` to the model ID you just configured. The API key is entered invisibly and is not embedded in the command text:

```
GEMINI_MODEL="MODEL_AVAILABLE_TO_YOUR_KEY"
read -rsp "Gemini API key: " GEMINI_KEY; echo
curl -sS --max-time 45 \
  -H "x-goog-api-key: $GEMINI_KEY" \
  -H "Content-Type: application/json" \
  -d '{"contents":[{"parts":[{"text":"Reply only OK"}]}]}' \
  "https://generativelanguage.googleapis.com/v1beta/models/${GEMINI_MODEL}:generateContent"
unset GEMINI_KEY GEMINI_MODEL
```

If this succeeds while the app ping fails, authentication and the model are accepted for the minimal request; investigate the app's full request payload and provider handling. If it fails, use the returned provider error to correct the key, model, or API settings.

The ping reports only a generic diagnostic on failure. Error Log and `logs/web.error.log` contain a status or network diagnostic, not necessarily Gemini's full error body. An HTTP 400 means Gemini rejected the request; a direct request using the same model and key can expose the provider's explanation. A 403 indicates authentication or API-key restrictions; 429 indicates rate limiting; 503 and timeouts indicate temporary service or network issues. Do not keep switching model IDs without checking the API response and the models available to the key.

Business-data consent for hosted models

In HW AI Support Settings, a System Manager must:

1. Enable Tools Enabled.
2. Choose Hosted LLM, aggregated results only as Business Data Mode.
3. Enable Hosted Consent after reviewing what data is sent.

The provider receives the user's question and tool schemas; after permission checks, it receives a compact result needed to summarize the answer. Hosted mode is intended to avoid sending detailed invoice rows. Turn off the tools or consent to disable hosted business-data answers. See supported tools and boundaries for the current data scope.

OpenAI-compatible provider

In HW AI Support Settings, set LLM Provider to `openai_compatible`, enter the endpoint and model in Local LLM Base URL and Local LLM Model, and select a matching Business Data Mode. For a local Ollama-compatible server, a typical base URL is `http://localhost:11434/v1`. It must be reachable from the Bench server, not merely from the user's browser.

For localhost endpoints, a key is optional. For a remote endpoint, configure `openai_api_key` in site config or `OPENAI_API_KEY` in the Bench server environment. For local-only handling, set Business Data Mode to Local LLM only. Enable tools separately when business insights are required.

Ollama in the same WSL environment as Bench

Install Ollama in Ubuntu/WSL, pull a small multilingual model, and start the server if the installer did not start it automatically. These commands run in a WSL terminal:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5:1.5b
ollama serve
```

Keep `ollama serve` running in its terminal. If Ollama is already running as a service, skip that command. The model download is about 1 GB. Ollama lists Qwen 2.5 as multilingual and tool-capable, but 1.5B is a small model: expect weaker retrieval-grounded answers and less reliable tool selection than larger models. Ollama install, Qwen 2.5 1.5B, Ollama tool support.

Then open HW AI Support Settings and set:

- LLM Provider: `openai_compatible`
- Local LLM Base URL: `http://localhost:11434/v1`
- Local LLM Model: `qwen2.5:1.5b`
- Business Data Mode: `Local LLM only`
- Tools Enabled: enabled only if you want Phase 2 read-only business questions

No API key is required for the local endpoint. Since Bench and Ollama are in the same WSL environment, `localhost` should refer to that Ollama service. Verify it with `ollama list`, then run the provider ping below. For basic Phase 1 tests, ask `What is a Sales Order?`; if enabling tools, test stock only with an item that exists and that your account can read.

Verify the connection with:

```
bench --site development.localhost clear-cache
bench --site development.localhost execute hw_ai_support.llm.ping
```

Troubleshooting checklist

1. Confirm LLM Provider in HW AI Support Settings matches the provider being configured.
2. Confirm the server-side key exists without printing it: `bench --site development.localhost show-config` may display secrets, so do not share its output. Prefer rotating and resetting the key if uncertain.
3. Confirm the model ID is available to that provider credential and the endpoint is reachable from the Bench host.
4. Run `bench --site development.localhost clear-cache`, then run the provider ping again.
5. If ping succeeds but a Phase 2 question fails, check the newest Error Log and verify tools/data mode/consent and user permissions.
6. If Gemini returns HTTP 400, test the same model and key with a direct API call to see the provider response body. The current app log records the status but not that body, and ping intentionally does not return raw provider response content. Do not share the API key.

Provider requests have a bounded timeout and use `hw_ai_max_output_tokens` from site config (default 1024). The chat API displays localized generic errors rather than exposing provider diagnostics to end users.